

# TEMPO: Cross-Layer Global Orchestration for Disaggregated LLM Inference on Shared HPC Fabrics

Anonymous Author(s)

## Abstract

Prefill/decode disaggregation removes one form of GPU interference but creates a coupled control problem: a local prefill consumes decode capacity, whereas a remote prefill consumes KV-transfer, receiver, and network capacity. The better path therefore changes with workload, cache residency, decoder service, fabric contention, and tenant objectives. Existing inference schedulers and network transports each expose valuable parts of this state, but optimizing one layer can move the bottleneck to another.

We present TEMPO, a global orchestrator that joins vLLM service state, LMCACHE KV semantics, Cassini/Slingshot endpoint telemetry, topology, and business SLO state in an identity-bound control loop. It atomically admits a request, selects a prefill/decode edge and local or remote execution, reserves decoder and transfer resources, and records fail-closed lifecycle evidence. Near-tied remote replicas are balanced by controller-owned source/edge service, and a node-pair-shard hierarchy bounds global fan-in.

On four Perlmutter nodes (16 A100 GPUs), using Qwen2.5-7B-Instruct, actual vLLM, LMCACHEConnectorV1, NIXL/UCX over CXI, and a fresh held-out workload, TEMPO attains all 210 offered foreground SLOs across normal, decoder-hot, and remote-favorable regimes. Relative to the strongest fixed policy, it reduces stressed p99 latency by 66.1–90.6%; relative to a predictor, by 61.7–93.4%. In a same-allocation post-hoc extension, TEMPO reduces p99 by 75.8–93.8% over a NetKV Algorithm-1 reproduction while improving stressed SLO attainment from 73/120 and 0/30 to 120/120 and 30/30. A 1,024-pair control-plane replay reduces global payload by 87.5%. We explicitly separate the independently validated four-node result from the post-hoc SOTA comparison and from CPU-only scale evidence.

## 1 Introduction

Large-language-model serving separates prompt prefill from autoregressive decode because their compute and memory behavior differ. Systems such as DistServe and Mooncake isolate the phases and move KV state between pools, improving resource planning and cache reuse [9, 7]. vLLM provides the high-throughput execution substrate [3], and LMCACHE exposes KV state as a

cross-engine storage and communication layer [2].

Disaggregation, however, does not eliminate contention. It changes where contention appears. Sending a request through a prefill node can queue behind long prompts and then pay an inter-node KV transfer. Deflecting prefill to a decode node avoids that transfer but competes with active decodes. Choosing a different decode receiver can relieve its GPU queue while creating incast or moving load to a congested fabric edge. In a shared HPC environment, inference also coexists with collective communication and jobs whose utility differs by tenant, deadline, and fairness obligation.

This creates a systems question broader than route prediction:

Can one controller combine application semantics, GPU service, KV movement, fabric state, topology, and business demand, then jointly control admission, placement, and execution without simply moving the bottleneck?

Network-aware selection, as formalized by NetKV [6], makes transfer cost visible. Kairos [1] instead exploits decode-side compute slack with a TBT-safe chunk schedule. Multipath transports such as MRC improve path utilization and recovery [8]. These are complementary mechanisms: none alone owns the complete request lifecycle from tenant admission through prefill/decode placement, KV transfer, receiver service, and completion.

TEMPO treats this cross-layer lifecycle as the unit of control. Its design is guided by three observations from our development history. First, local and remote winners reverse as decoder and transfer pressure change. Second, completed-only latency is misleading when a policy rejects or loses requests. Third, telemetry is useful only when its identity, freshness, support status, and actuation are bound to the request that consumed it.

This paper makes four contributions:

1. We formulate disaggregated inference on shared HPC fabrics as a cross-layer global admission and edge-selection problem, including tenant utility and failure state rather than latency prediction alone.
2. We implement an identity-bound state and actuation plane spanning vLLM, LMCACHE/NIXL, Cassini/Slingshot, topology, and business policy. The same transaction selects local/remote execution and physical  $P_i-D_j$  placement, reserves resources, and

closes the stream lifecycle.

3. We construct realistic matched workloads that separately expose decoder-hot and remote-favorable operation, and evaluate offered-population SLO attainment, tail latency, background utility, fairness, route diversity, and control overhead on an actual four-node Perlmutter deployment.
4. We provide a fresh held-out native result, paper-policy comparison, and source-bound hierarchy receipt with explicit claim boundaries and preserved failure artifacts.

## 2 Background and Motivation

### 2.1 Disaggregation moves the bottleneck

A request with prompt length  $n$  and output length  $m$  can use one of two broad paths. A remote path computes prefill on  $P_i$ , transfers the KV cache, and decodes on  $D_j$ . Its TTFT includes prefill queueing and service, KV movement, receiver queueing, and the first decode step. A local-deflection path performs chunked prefill on  $D_j$  and removes inter-node KV movement, but each chunk inflates a mixed prefill/decode step. Thus

$$\hat{T}_{\text{remote}} = T_{P_i}^{q+s} + T_{i \rightarrow j}^{KV} + T_{D_j}^{q+1}, \quad (1)$$

$$\hat{T}_{\text{local}} = T_{D_j}^q + T_{D_j}^{\text{chunk+decode}}. \quad (2)$$

The hidden coupling is that  $T_{i \rightarrow j}^{KV}$  depends on source and receiver traffic, transport inflight work, NIC state, and competing communication, while the local term depends on active sequences, KV occupancy, and the TBT slack of existing decodes. A decision made from one term can be correct at admission and harmful by execution unless resources and lifecycle are reserved consistently.

### 2.2 Why shared HPC is a distinct setting

Perlmutter GPU nodes contain four A100 GPUs and four Cassini Slingshot 11 NICs, each rated at 200 Gb/s (25 GB/s), on a three-hop dragonfly fabric [4, 5]. This gives substantial aggregate bandwidth but also several independently changing contention domains: GPU compute and HBM, PCIe and host service, NIC traffic classes, inter-group links, and collective communication. A scheduler cannot infer the active bottleneck from “local” or “remote” alone.

The production objective is also not one scalar latency. An interactive tenant may require protected TTFT and end-to-end service; batch/background work must retain bounded service; and a failure should release every reservation. We therefore evaluate good requests out of the *offered* population, tenant fairness, and terminal outcomes, not only quantiles among completions.

### 2.3 Motivating failure of partial policies

Our held-out workload contains a decoder-hot regime and a P-only remote-favorable regime. The former overloads decode service with simultaneous local and remote pressure. The latter pre-seeds exact 4,094-token source hits and loads both decoders, making remote source choice valuable. No single static path wins both. In the final fresh run, the strongest fixed and predictor policies exhibit 9.8–50.6 second stressed p99 latency, whereas a joint policy stays near 3.3 seconds. This gap is not produced by an isolated network microbenchmark; it arises from the interaction of offered requests, actual model execution, KV movement, decoder service, and admission.

## 3 System Model and Objective

Let  $\mathcal{P}$  and  $\mathcal{D}$  be prefill and decode instances. A request  $r$  has tenant, arrival, deadline, prompt/cache identity, output demand, and a candidate set  $\mathcal{E}_r$ . A candidate  $e = (p, d, \rho)$  chooses source  $p$ , decoder  $d$ , and route  $\rho \in \{\text{local}, \text{remote}\}$ . Each candidate carries a resource vector

$$\mathbf{w}_e = (w_{\text{decode}}, w_{\text{local prefill}}, w_{KV}, w_{\text{remote prefill}}, w_{\text{semantic}}). \quad (3)$$

The controller consumes an atomic telemetry batch  $\mathbf{s}$  containing vLLM running/waiting requests and KV occupancy, cache-residency evidence, LMCash semantic/byte inflight state, supported Cassini signals, endpoint health, and controller-owned reservations. Unsupported signals are unknown, not zero.

Conceptually, TEMPO chooses a feasible candidate that minimizes predicted service plus externality and business debt:

$$e^* = \arg \min_{e \in \mathcal{E}_r \cap \mathcal{F}(\mathbf{s})} \{ \hat{T}_e + \lambda_f C_e^{\text{fabric}} + \lambda_q C_e^{\text{service}} + \lambda_b C_{r,e}^{\text{business}} \}. \quad (4)$$

$\mathcal{F}$  enforces cache, capacity, freshness, deadline, health, and transport guards. The implementation uses calibrated service priors, resource envelopes, tenant reservations, and causal receipts rather than assuming that this expression is an exact latency oracle. Requests can be admitted, deferred in a bounded queue, or rejected. Every admitted request owns a lease that is released on first response, completion, explicit route failure, timeout, or abort.

## 4 Design

### 4.1 Identity-bound state plane

Node agents export scheduler and endpoint state; pair agents bind it to model, engine, topology, epoch, and

monotonically increasing sequence identities. A request-triggered collector obtains one all-pair batch with bounded timeout and one validation-only retry. The coordinator atomically installs only a complete and causally valid batch. A stale fallback is permitted only within a tenant-specific grace and still passes resource and health guards.

Cross-layer signals retain five fields: value, unit, support status, source, and sample identity. This matters on managed HPC systems where some counters are available through user-visible sysfs or endpoint APIs and others require operator integration. Fabric-unavailable must not silently become fabric-idle.

## 4.2 Global candidate and atomic commit

For each request, TEMPO constructs feasible local and remote candidates over the physical  $P \times D$  mesh. Cache identity determines whether a source can supply a full hit, partial reuse, or a confirmed miss. The coordinator scores candidates, admits or queues the request, and returns an immutable commit that contains route, source, decoder, edge, telemetry sequence, resource bindings, and optional joint actuation.

The frontend reserves decoder placement from that commit and sends a remote request to the committed prefill source. The pair router independently checks the commit before invoking vLLM/LMCache. A mismatch fails closed. This separation prevents the common evidence bug of confusing the committed decode destination with the prefill source on a cross-pair edge.

## 4.3 Business admission and fairness

Protected tenants receive bounded decoder admission and service-lane capacity; background tenants receive explicit limits, maximum wait, and starvation escape. Fairness and capacity are part of candidate feasibility before latency tie-breaking. A request that cannot be safely admitted appears as a global reject in the offered-population metric rather than disappearing from a completed-only distribution.

The policy also tracks controller-owned virtual service. Among remote candidates whose calibrated scores differ only within telemetry uncertainty and that have equivalent cache and business semantics, TEMPO selects the source and edge with lower virtual finish. It never forces a materially worse edge to satisfy a quota. This breaks deterministic source ties while retaining the original service optimum and yields causal source-balance receipts.

## 4.4 Cross-layer actuation

An admitted commit can carry route-specific resource limits, transfer semantic concurrency, receiver staggering, and a shared-fabric budget. Critical transport or endpoint failures remain hard guards; non-critical pressure is priced against the resource envelope. Local deflection is available

when remote movement is harmful, while remote execution remains available when decoder-local prefill would consume scarce service.

This mechanism is intentionally broader than a network penalty. The same decision can respond to decoder running-set inflation, a full remote cache hit, fabric pressure, protected business demand, and pair health. The action and the signals that justified it are recorded together.

## 4.5 Hierarchical fan-in

At large logical pair counts, pair agents locally rank their route candidates. Shard reducers forward a bounded frontier and an omission receipt; a global coordinator retains final policy ownership. The request identity, telemetry epoch, omitted-pair count, and candidate fingerprint cross all levels. This bounds wire payload and global candidate count, but it does not by itself prove native inference scalability; Section 7.5 keeps that boundary explicit.

# 5 Implementation

The prototype comprises approximately four functional planes.

**Serving carrier.** Four vLLM 0.26.0 engines run Qwen2.5-7B-Instruct at tensor parallelism four. Two act as prefill producers and two as decode consumers. The common remote path is LMCacheConnectorV1 with NIXL/UCX over the CXI provider. Local execution uses vLLM chunked prefill on the decoder. The experiment disables decoder prefix caching so LMCache source evidence and local work are not conflated.

**State and control.** The Python implementation contains strict dataclasses for resource vectors, pair telemetry, cross-layer signals, route candidates, decisions, failure reports, and hierarchy receipts. An asynchronous coordinator joins telemetry collection, global admission, stream feedback, and release. Frontend and pair router APIs preserve request IDs and commitment hashes across HTTP and SSE boundaries.

**Perlmutter instrumentation.** Cassini readers expose per-NIC and traffic-class counters with explicit support classification. vLLM scheduler snapshots supply running, waiting, and KV occupancy. LMCache supplies semantic and byte-inflight state. NCCL observer schemas exist and are exercised in opt-in co-job tests, but NCCL latency signals were unavailable in the final held-out headline and are reported as such.

**Failure discipline.** The system rejects wrong epoch, topology, sequence, hash, stale state, missing candidate ownership, duplicate lifecycle events, and leaked leases. The artifact retains unsuccessful launches and validator failures. Experiments run inside a four-hour no-shell interactive allocation; compute occurs only in foreground

Slurm steps.

## 6 Evaluation Methodology

We ask five questions:

1. Does TEMPO improve offered-population SLO attainment and tail latency under decoder-hot contention?
2. Does it retain remote execution and use the physical mesh when exact source reuse makes remote favorable?
3. Does foreground protection preserve bounded background utility and tenant fairness?
4. How does it compare with paper-derived network-aware selection and prefill deflection on the same actual carrier?
5. Does the hierarchy bound global fan-in as logical pair count grows?

### 6.1 Testbed and workload

The native testbed is one Perlmutter allocation with four GPU nodes and 16 A100 GPUs. Each node contributes one TP4 engine. Perlmutter uses Slingshot 11 with four Cassini NICs per GPU node [4]. We use an 8,192-token model limit, 16 maximum sequences per engine, 3,000 ms TTFT, 150 ms TBT/TPOT, and 8,000 ms end-to-end SLOs.

The held-out contract fixes seed 20260825, deterministic sub-spacing arrival jitter, block order, prompt namespace transform, request counts, profile and source hashes before the fresh execution. The three analysis regimes are:

- **normal**: 60 foreground requests across two control blocks;
- **miss-hot**: 120 foreground requests plus simultaneous remote and decoder-local background pressure across both decode destinations;
- **remote-favorable**: 30 foreground requests with exact 4,094-token P-only source hits while 1,344 background requests load both decoders.

We report offered, completed, and SLO-good requests; TTFT, TPOT and end-to-end p50/p95/p99 among completions; route and edge counts; background terminal outcomes; per-tenant completion and Jain fairness; telemetry overhead and support; and all failure/rejection counts.

### 6.2 Baselines

The independently validated matrix contains local and remote fixed policies, a calibrated predictor, and queue/GPU observe-only routing. All use the same model, engines, request population, and remote connector.

The C10 extension adds two paper-policy reproductions. **NetKV** uses remote candidates and minimizes KV transfer plus decoder queue and first-step cost, using 25 GB/s per TP shard, supported Cassini congestion, LMCACHE self-inflight, and vLLM scheduler state [6]. **Kairos-X512** uses  $\alpha = 1.3$ , a 0.9 TBT safety margin, and an actual

Table 1: Fresh four-node C9 held-out result. Counts are offered/completed/SLO-good.

| Regime           | Count       | E2E p50 | E2E p99 |
|------------------|-------------|---------|---------|
| Normal           | 60/60/60    | 2.910 s | 3.150 s |
| Miss-hot         | 120/120/120 | 3.058 s | 3.337 s |
| Remote-favorable | 30/30/30    | 3.067 s | 3.357 s |

Table 2: TEMPO E2E p99 reduction relative to matched baselines.

| Baseline        | Miss-hot | Remote-favorable |
|-----------------|----------|------------------|
| Strongest fixed | 66.12%   | 90.55%           |
| Predictor       | 61.69%   | 93.36%           |
| Queue/GPU-only  | 58.48%   | 93.54%           |

512-token decoder batching ceiling [1]. Stock vLLM lacks Kairos’s per-request dynamic chunk schedule, and the reported repository was unavailable at our freeze time; this is therefore a declared  $X = \{512\}$  subset, not the authors’ full implementation.

Both paper baselines disable TEMPO’s business priority, fairness reservation, pair scaling, shared-fabric budget, and stagger. A compatibility adapter emits frozen-workload lifecycle evidence but never waits or throttles. The analyzer verifies `policy_effect=none` and `wait_ns=0` for every receipt.

### 6.3 Claim boundaries

The C9 headline is a fresh-allocation, one-shot, source-bound independent validation. C10 is an actual-system, same-population *post-hoc* extension because its adapters were frozen after that allocation began. The hierarchy experiment is CPU control-plane evidence only. We do not claim a full Kairos reproduction, an independent C10 SOTA result, native scale beyond four nodes, or supported NCCL latency telemetry in the held-out headline.

## 7 Evaluation

### 7.1 Fresh held-out native result

Table 1 reports TEMPO on all three regimes. Every offered foreground request completes within all SLOs. The remote-favorable phase uses 29 remote requests and one local request while retaining a 3.36-second p99.

Table 2 shows relative p99 reduction. The strongest fixed arm is selected per stressed regime by the frozen analyzer, rather than chosen after seeing TEMPO. The improvement is large because partial policies either queue behind the hot decoder, retain an expensive remote path, or fail to coordinate admission with execution.

## 7.2 Background utility and control cost

Foreground protection does not delete the offered background population. In miss-hot blocks, background completes 1,204 of 1,404 requests (85.75%); the minimum block-by-tenant completion is 76.50%, the two-tenant Jain fairness is 0.9979, and service-lane failures are 0.93%. In remote-favorable, background completes 1,344/1,344. These outcomes include explicit rejects and failures in the denominator.

Request-triggered telemetry collection has 28.62 ms p50 and 132.42 ms p99; admission has 29.46 ms p50 and 133.26 ms p99. All 30 remote-favorable decisions carry explicit support classifications. Cassini signals are supported for 29 and LMCache inflight signals for all 30. The source/edge virtual-service binding appears on 29 decisions. Unavailable NCCL and LMCache latency signals remain unavailable.

## 7.3 Physical mesh use

Across regimes, TEMPO uses both local decoder edges and all four remote edges:  $P_0 \rightarrow D_0$ ,  $P_0 \rightarrow D_1$ ,  $P_1 \rightarrow D_0$ , and  $P_1 \rightarrow D_1$ . This rules out a degenerate “global” controller that always sends requests to one source or one receiver. Every cross-pair remote completion carries a commit whose source, destination, edge identity, cache evidence, and telemetry sequence are consistent.

## 7.4 Paper-policy comparison

Table 3 compares actual-system C10 results. NetKV is strong in the normal phase, where it completes all requests and trails TEMPO p99 by 4.54%. Under miss-hot contention it completes 117/120 but only 73 meet the SLO; under remote-favorable load it admits one request, whose 53.9-second latency misses the SLO. TEMPO reduces NetKV p99 by 75.79% and 93.77% in the two stressed regimes and increases SLO-good by 47 and 30.

Kairos-X512 completes half of normal requests, rejects every miss-hot victim, and completes one remote-favorable victim. Because miss-hot has zero completions, its latency quantiles are undefined. We report a 120-request completion/SLO advantage, not a fabricated infinite latency speedup. On the defined remote tail, TEMPO reduces p99 by 30.82%.

The comparison demonstrates the motivating point, not that the referenced systems lack value. Network-aware remote selection cannot rescue a request when the correct action is decoder-local prefill or business-aware admission; a fixed safe deflection subset cannot exploit all time-varying decode slack or cache/fabric alternatives. TEMPO retains both action families and commits them with shared state.

## 7.5 Control-plane scale

We replay the same two-route candidate population at 2, 8, 32, 128, 512, and 1,024 logical pairs for 15 repetitions. Pair ownership is prepartitioned, so pair-agent timing measures local ranking and receipt construction rather than repeatedly scanning the global population.

At 1,024 pairs, the bounded path forwards 256 of 2,048 candidates and records 896 omitted pairs. Serialized global payload falls from 666,815 to 83,358 bytes (87.50%). Full scan p50/p99 is 49.74/152.84 ms; pair-agent p50 is 29.65 ms; bounded-global p50 is 55.43 ms; and bounded end-to-end p50/p99 is 85.33/158.24 ms. Thus global payload and candidate fan-in are bounded, while this single-process Python replay is not faster than full scan. Distributed wall-clock and native utility beyond four nodes remain future work.

## 8 Failure Analysis and Reproducibility

The final result follows hundreds of preserved development versions rather than hiding unsuccessful candidates. Several route-only and threshold policies improved completed-only latency by rejecting too much work; they were discarded by offered-population gates. Other runs exposed stale telemetry, source/destination evidence aliasing, and frontend lifecycle failures.

C10 itself produced two non-performance failures. The first baseline adapter removed a required evidence receipt while correctly removing policy. The fix emits a no-effect lifecycle receipt and the analyzer proves zero wait and no throttle. The second failure occurred after NetKV executed a valid  $P_0 \rightarrow D_1$  request: a legacy validator treated the committed decoder destination as the prefill source. A mesh-aware adapter now validates commit source, decoder, and edge separately without changing the workload or score. The successful Kairos result was hash-bound and reused; only NetKV was executed under the corrected evidence source.

The artifact includes run contracts, source inventories, raw request and router ledgers, endpoint evidence, logs, failed result roots, analyses, and SHA-256 digests. Independent and post-hoc results are stored separately.

## 9 Related Work

**LLM serving and disaggregation.** vLLM reduces KV-memory waste and enables flexible batching with PagedAttention [3]. DistServe separates prefill and decode to optimize goodput under phase-specific SLOs [9]. Mooncake builds a KV-centric disaggregated cache and global scheduler at production scale [7]; LMCache exposes reusable, movable KV state across engines [2]. TEMPO uses vLLM and LMCache as its actual carrier and focuses

Table 3: Actual four-node C10 paper-policy comparison. SLO is good/offered; latency is completion E2E p99. Kairos is the declared  $X = \{512\}$  subset.

| Policy             | Normal |         | Miss-hot |           | Remote-favorable |          |
|--------------------|--------|---------|----------|-----------|------------------|----------|
|                    | SLO    | p99     | SLO      | p99       | SLO              | p99      |
| TEMPO              | 60/60  | 3.150 s | 120/120  | 3.337 s   | 30/30            | 3.357 s  |
| Kairos- $X512$     | 30/60  | 3.223 s | 0/120    | undefined | 1/30             | 4.853 s  |
| NetKV reproduction | 60/60  | 3.300 s | 73/120   | 13.780 s  | 0/30             | 53.914 s |

on a shared-HPC control loop that joins their state with fabric, topology, failure, and tenant policy.

**Network-aware routing.** NetKV introduces a network cost oracle and an  $O(|D|)$  decode-instance selector that combines KV transfer and decoder cost [6]. Its published evaluation uses a 64-GPU four-tier fat-tree simulator driven by Mooncake traces. TEMPO extends the action space to local prefill, admission, business fairness, and joint resource commitment, and evaluates a NetKV policy reproduction on an actual Slingshot carrier. MRC provides production-grade multipath, congestion control, loss recovery, and path resilience at the transport layer [8]. Such transport mechanisms can improve the available paths beneath TEMPO but do not replace inference-semantic and business orchestration.

**Prefill deflection.** Kairos estimates regular prefill TTFT and searches decode-node chunk schedules that preserve TBT SLO, reporting large P95 TTFT and SLO gains on a 2P2D A100 cluster [1]. TEMPO likewise includes decoder-local chunked prefill, but decides it jointly with remote cache/fabric candidates and global admission. Our  $X512$  experiment is deliberately narrower than Kairos and is used only as a same-carrier mechanism comparison.

## 10 Limitations

The independently validated native experiment uses four nodes and one model. It proves a four-node Perlmutter mechanism and performance result, not facility-wide production superiority. The 1,024-pair result is CPU-only and does not measure distributed agent wall-clock, NIC traffic, GPU inference, or failure convergence.

The final held-out run has supported Cassini and LMCache inflight state but not NCCL collective latency or LMCACHE transfer-tail signals. Those schemas and co-job mechanisms exist, yet a future source-frozen experiment must make them supported and perform a causal ablation before claiming that those signals drive the headline gain.

C10 is post-hoc and must be repeated unchanged on a fresh allocation for an independent SOTA-extension claim. Kairos- $X512$  is not full Kairos. Results may also change for larger models, longer contexts, more cache tiers, different topologies, and operator-level scheduling privileges. At user scope, TEMPO controls only its allocation and treats other jobs as exogenous; it does not modify facility policy or other users’ workloads.

## 11 Conclusion

Disaggregated LLM inference on a shared HPC fabric is not a binary local-versus- remote routing problem. Decoder service, KV movement, receiver load, fabric state, cache identity, tenant utility, and failures form one coupled lifecycle. TEMPO makes that lifecycle the unit of orchestration. On an actual four-node Perlmutter deployment, a fresh held-out run sustains 100% foreground SLO attainment and roughly 3.3-second stressed p99 while matched partial policies degrade to 8–53 seconds or reject work. Background and telemetry gates show that the gain is not obtained by silently dropping the rest of the population. Post-hoc paper-policy and logical-scale results strengthen the motivation while preserving their narrower claim boundaries. The next step is an unchanged independent paper-policy rerun and native scale beyond four nodes, not another threshold search on the proven workload.

## References

- [1] Shrikara Arun, Anjaly Parayil, Srikant Bharadwaj, Renee St. Amant, and Victor Rühle. Towards load-aware prefill deflection for disaggregated LLM serving, 2026.
- [2] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. LMCACHE: An efficient KV cache layer for enterprise-scale LLM inference, 2025.
- [3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [4] National Energy Research Scientific Computing Center. Perlmutter architecture. NERSC Documentation, 2026. Accessed 2026-08-25.
- [5] National Energy Research Scientific Computing Center. Perlmutter slingshot interconnect. NERSC Documentation, 2026. Accessed 2026-08-25.

- [6] Mubarak Adetunji Ojewale. NetKV: Network-aware decode instance selection for disaggregated LLM inference, 2026.
- [7] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST)*, pages 155–170, 2025.
- [8] Rip Sohan, Eric Spada, Eric Davis, Mark Handley, Idan Burstein, Tony Hurson, Jithin Jose, Vivek Kashyap, Rong Pan, Sayantan Sur, et al. The multi-path reliable connection (MRC) transport, 2026.
- [9] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 193–210, 2024.